

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

API con Persistencia en PostgreSQL usando Docker

### 01 . Práctica de laboratorio

---

## TODO API con Persistencia en PostgreSQL usando Docker

|                       |                                               |
|-----------------------|-----------------------------------------------|
| Cátedra               | Programación Avanzada                         |
| Unidad temática       | Persistencia de datos y contenedores (Docker) |
| Práctica base         | TODO API en memoria (Nivel Básico)            |
| Nivel                 | Intermedio                                    |
| Modalidad             | Individual                                    |
| Entorno requerido     | Node.js 20+, Docker y Docker Compose          |
| Versión del documento | 1.0                                           |

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## Índice

---

1. Resumen ejecutivo (Overview)
  2. Contexto y planteamiento del problema
  3. Objetivos
  4. Público objetivo y prerequisites
  5. Requisitos funcionales
  6. Requisitos no funcionales
  7. Diseño técnico / Arquitectura
  8. Especificación de la API
  9. Consigna: actividades a realizar
  10. Criterios de aceptación (Definition of Done)
  11. Entregables
  12. Rúbrica de evaluación
  13. Cronograma sugerido
  14. Riesgos y supuestos
  15. Recursos y referencias
- Anexo A: Solución de problemas frecuentes

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 2. Resumen (Overview)

---

Esta práctica extiende el trabajo realizado en la Práctica 01 ("TODO API", nivel básico), donde se construyó una API REST con Express que administraba tareas en un arreglo en memoria. La limitación de esa versión es evidente: al reiniciar el servidor, toda la información se pierde.

En esta segunda práctica, el estudiante va a reemplazar el almacenamiento en memoria por una base de datos PostgreSQL real, y va a orquestar tanto la API como la base de datos como contenedores Docker independientes que se comunican entre sí mediante Docker Compose. El resultado es una arquitectura de dos servicios, reproducible en cualquier máquina con Docker instalado, sin importar el sistema operativo.

Este documento describe la práctica con el formato de un PRD (Product Requirements Document) adaptado a un contexto académico: define el problema, los objetivos, los requisitos funcionales y no funcionales, el diseño técnico esperado, la consigna paso a paso, los criterios de aceptación y la rúbrica de evaluación.

## 3. Contexto y planteamiento del problema

---

Las aplicaciones reales casi nunca guardan su información en memoria: necesitan persistirla en una base de datos para que sobreviva a reinicios, se pueda consultar de forma concurrente y escale más allá de un único proceso. Además, en un equipo de trabajo real, cada desarrollador/a necesita poder levantar el mismo entorno (API + base de datos) de manera idéntica, sin depender de instalar PostgreSQL manualmente en su sistema operativo.

Problema a resolver en esta práctica:

- Los datos de la Práctica 01 no persisten entre reinicios del servidor.
- No existe un mecanismo estandarizado para levantar el proyecto junto con su base de datos en cualquier máquina.
- El código actual no protege contra inyección SQL porque no accede a ninguna base de datos todavía.

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 4. Objetivos

---

### 4.1 Objetivo general

Que el/la estudiante sea capaz de contenerizar una aplicación Node.js y una base de datos PostgreSQL, conectarlas mediante Docker Compose, y migrar una API CRUD desde almacenamiento en memoria hacia persistencia real usando consultas SQL parametrizadas.

### 4.2 Objetivos específicos

- Levantar un entorno multi-contenedor con Docker Compose (API + base de datos).
- Explicar la diferencia entre resolución de nombres dentro de la red de Compose (nombre del servicio) y en el host local (localhost).
- Diseñar un esquema relacional simple (tabla tasks) y poblarlo mediante un script de inicialización.
- Utilizar el driver pg de Node.js con consultas parametrizadas para prevenir inyección SQL.
- Completar los endpoints de actualización (PUT) y eliminación (DELETE) siguiendo el patrón ya usado en los endpoints existentes.
- Verificar de forma empírica que los datos persisten frente a un reinicio del contenedor de la API, y que se pierden solo si se elimina el volumen de la base de datos.

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

#### 4.3 Fuera de alcance (Non-goals)

Para mantener el foco pedagógico de esta práctica, quedan explícitamente fuera de alcance:

- Autenticación y autorización de usuarios.
- Uso de un ORM (Sequelize, Prisma, TypeORM); se trabaja con SQL directo a través del driver pg.
- Migraciones versionadas de esquema (se usa un único script init.sql).
- Despliegue en un proveedor cloud (AWS, GCP, Azure); el alcance es local, con Docker Compose.
- Tests automatizados (unitarios o de integración); pueden proponerse como práctica adicional.

## 5. Público objetivo y prerequisites

---

Esta práctica está dirigida a estudiantes que ya completaron la Práctica 01 y cuentan con los siguientes conocimientos y herramientas:

- JavaScript moderno (ES Modules, async/await) y fundamentos de Express.js.
- Conceptos básicos de SQL: SELECT, INSERT, UPDATE, DELETE.
- Docker Desktop (o Docker Engine + Docker Compose) instalado y funcionando.
- Un cliente HTTP para pruebas: curl, Postman, Insomnia o Thunder Client.

## Licenciatura en Sistemas de Información

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 6. Requisitos funcionales

La tabla siguiente detalla el comportamiento esperado de la API, identificado con un código de trazabilidad (RF-0X), útil para referenciarlo durante la corrección.

| ID    | Requisito            | Descripción                                                                                                                    | Estado        |
|-------|----------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------|
| RF-01 | Listar tareas        | GET /tasks devuelve todas las tareas almacenadas en la base, ordenadas por id, con filtro opcional por status.                 | Implementado  |
| RF-02 | Obtener tarea por ID | GET /tasks/:id devuelve la tarea solicitada o 404 si no existe.                                                                | Implementado  |
| RF-03 | Crear tarea          | POST /tasks valida que title esté presente, inserta la fila en la base y responde 201 con el registro creado.                  | Implementado  |
| RF-04 | Actualizar tarea     | PUT /tasks/:id actualiza solo los campos recibidos en el body, sin pisar los que no vengan, y responde 404 si el id no existe. | A implementar |
| RF-05 | Eliminar tarea       | DELETE /tasks/:id elimina la fila y responde 204, o 404 si no existía.                                                         | A implementar |
| RF-06 | Filtrado por estado  | GET /tasks?status=pending filtra las tareas por el campo status.                                                               | Implementado  |

## Licenciatura en Sistemas de Información

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 7. Requisitos no funcionales

| Categoría                          | Requisito                                                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Persistencia                       | Los datos deben sobrevivir al reinicio del contenedor de la API, almacenados en un volumen Docker nombrado.                       |
| Portabilidad                       | El proyecto debe levantarse con un único comando (docker compose up --build) en cualquier sistema operativo con Docker instalado. |
| Seguridad                          | Todas las consultas SQL deben ser parametrizadas (\$1, \$2, ...); no se admite concatenación de strings en las queries.           |
| Disponibilidad / orden de arranque | La API no debe intentar conectarse a la base antes de que esta esté lista (healthcheck + depends_on: condition: service_healthy). |
| Configuración                      | Ninguna credencial ni host debe estar hardcodeado en el código: todo se define por variables de entorno (.env).                   |

## Licenciatura en Sistemas de Información

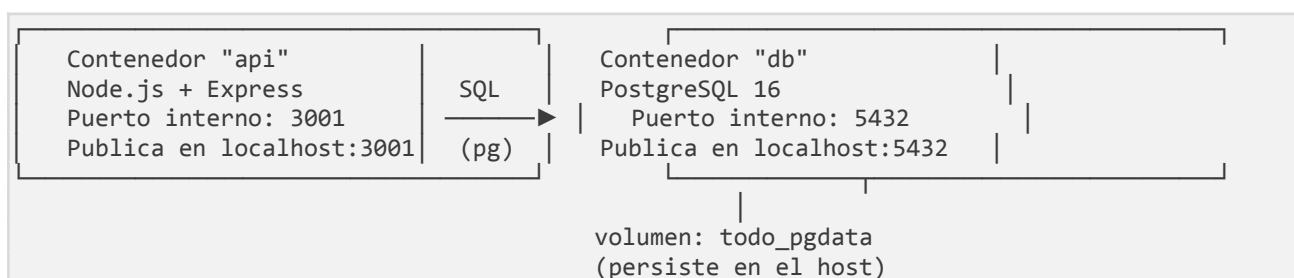
Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 8. Diseño técnico / Arquitectura

### 8.1 Arquitectura de contenedores

La solución se compone de dos servicios definidos en docker-compose.yml, conectados por una red interna que Compose crea automáticamente:



Punto clave para el aprendizaje: dentro de la red de Docker Compose, la API se conecta a la base usando el nombre del servicio (db) como host — NUNCA localhost, ya que localhost dentro de un contenedor apunta al propio contenedor, no al host ni a los demás servicios.

### 8.2 Modelo de datos

La tabla tasks se crea automáticamente la primera vez que se inicializa el volumen de Postgres, mediante el script db/init.sql:

| Columna     | Tipo                            | Descripción                                             |
|-------------|---------------------------------|---------------------------------------------------------|
| id          | SERIAL<br>PRIMARY<br>KEY        | Identificador autoincremental.                          |
| title       | VARCHAR<br>(255)<br>NOT<br>NULL | Título de la tarea, obligatorio.                        |
| description | TEXT                            | Descripción opcional, cadena vacía por defecto.         |
| status      | VARCHAR<br>(50)                 | "pending" o "completed"; por defecto "pending".         |
| due_date    | DATE                            | Fecha límite, opcional.                                 |
| created_at  | TIMESTAMP                       | Se completa automáticamente al insertar.                |
| updated_at  | TIMESTAMP                       | Debe actualizarse en cada UPDATE (parte del ejercicio). |

## Licenciatura en Sistemas de Información

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 9. Especificación de la API

| Método | Ruta       | Body                                        | Respuesta         | Códigos   |
|--------|------------|---------------------------------------------|-------------------|-----------|
| GET    | /tasks     | —                                           | Array de tareas   | 200       |
| GET    | /tasks/:id | —                                           | Objeto tarea      | 200 / 404 |
| POST   | /tasks     | { title, description?, status?, dueDate? }  | Tarea creada      | 201 / 400 |
| PUT    | /tasks/:id | { title?, description?, status?, dueDate? } | Tarea actualizada | 200 / 404 |
| DELETE | /tasks/:id | —                                           | Sin contenido     | 204 / 404 |

Nota: las filas PUT y DELETE corresponden a los requisitos RF-04 y RF-05, que el/la estudiante debe implementar como parte de la consigna (ver Sección 9).

## Licenciatura en Sistemas de Información

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 10. Consigna: actividades a realizar

---

### 10.1 Preparación del entorno

1. Cloná o copiá el proyecto base entregado (carpeta 02-todo-api-postgres/).  
<https://github.com/erneledesma/02-todo-api-postgres>
2. Copiá las variables de entorno: `cp .env.example .env`
3. Levantá los servicios: `docker compose up --build`
4. Verificá que la API responde en `http://localhost:3001/tasks` y que devuelve las dos tareas de ejemplo cargadas por `init.sql`.

### 10.2 Ejercicio 1 — Implementar PUT /tasks/:id (RF-04)

Completá el handler en `server.js`. La actualización debe conservar los valores existentes en los campos que no vengan en el body, usando COALESCE en la sentencia UPDATE, y debe actualizar `updated_at` a la fecha/hora actual. Si el id no existe, la respuesta debe ser 404.

### 10.3 Ejercicio 2 — Implementar DELETE /tasks/:id (RF-05)

Completá el handler de eliminación usando `DELETE ... RETURNING *` para poder distinguir si la fila existía. Respuesta 204 sin cuerpo si se eliminó, 404 si no existía.

### 10.4 Verificación de persistencia

1. Creá una tarea nueva con POST.
2. Reiniciá solo el contenedor de la API: `docker compose restart api`. Confirmá que la tarea sigue existiendo.
3. Ejecutá `docker compose down -v` y volvé a levantar el proyecto. Confirmá que esta vez sí se perdieron los datos, porque se eliminó el volumen.

### 10.5 Ejercicios bonus (opcionales)

- Agregar validación de status: solo aceptar "pending" o "completed", devolviendo 400 en cualquier otro valor.
- Agregar paginación simple a GET /tasks (parámetros limit y offset).
- Migrar el Dockerfile de la API a un esquema multi-stage build.

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 11. Criterios de aceptación (Definition of Done)

---

Se considera completa la práctica cuando se cumple, sin excepción, lo siguiente:

- `docker compose up --build` levanta ambos servicios sin errores en una máquina limpia.
- Los cinco endpoints (GET, GET/:id, POST, PUT, DELETE) responden con los códigos de estado correctos, incluyendo los casos 404.
- Ninguna consulta SQL concatena valores del usuario directamente en el string (se usan parámetros \$1, \$2, ...).
- Los datos persisten frente a un `docker compose restart api` y se pierden únicamente con `docker compose down -v`.
- El campo `updated_at` se actualiza correctamente al ejecutar un PUT.

## 12. Entregables

---

- Repositorio (zip o enlace Git) con el código fuente completo de la práctica, incluyendo `Dockerfile`, `docker-compose.yml` y `db/init.sql`.
- Captura de pantalla o log de consola mostrando `docker compose up --build` ejecutándose sin errores.
- Evidencia (capturas o colección de Postman/Insomnia exportada) de las pruebas de los 5 endpoints, incluyendo al menos un caso 404.
- Breve informe (media carilla) describiendo qué pasó con los datos en la prueba de persistencia de la Sección 9.4 y por qué.

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## 13. Riesgos y supuestos

---

- Se asume que Docker Desktop (o equivalente) está correctamente instalado y en ejecución antes de comenzar.
- Riesgo: el puerto 5432 puede estar ocupado por una instalación local de Postgres; solución documentada en el Anexo A.
- Riesgo: si se edita el código sin volver a construir la imagen (docker compose up --build), los cambios no se reflejan.
- Supuesto: el/la estudiante ya aprobó o cursó la Práctica 01 y comprende el mapeo entre métodos HTTP y operaciones CRUD.

## 14. Recursos y referencias

---

- Documentación oficial de Docker Compose: <https://docs.docker.com/compose/>
- Documentación oficial de PostgreSQL: <https://www.postgresql.org/docs/>
- Driver node-postgres (pg): <https://node-postgres.com/>
- Documentación de Express.js: <https://expressjs.com/>

## **Licenciatura en Sistemas de Información**

Programación Avanzada - 2026

### API con Persistencia en PostgreSQL usando Docker

## Anexo A: Solución de problemas frecuentes

---

### El puerto 5432 ya está en uso

Si hay una instancia de Postgres corriendo localmente, cambiar el mapeo de puertos en `docker-compose.yml`, por ejemplo a "5433:5432".

### La API intenta conectarse antes de que la base esté lista

El healthcheck del servicio db junto con `depends_on: condition: service_healthy` en el servicio api están pensados para evitar esta condición de carrera. Si igual ocurre, volver a ejecutar `docker compose up`.

### Los cambios en `server.js` no se reflejan

La imagen se construye una sola vez. Después de modificar el código hay que volver a ejecutar `docker compose up --build`.

### `init.sql` no crea la tabla

Ese script corre una única vez, al crear el volumen. Si el proyecto ya se había levantado antes, hay que eliminar el volumen (`docker compose down -v`) y volver a levantar los servicios.